

Documento de Requisitos

Equipe

Nome	Função
Ediene Gomes de Lima	Engenheiro de Requisitos
Jamili Martins de Oliveira	Engenheiro de Requisitos
Lucas Albuquerque Teixeira	Engenheiro de Requisitos
Natália dos Santos Nascimento	Engenheiro de Requisitos

Controle de Versão

Autor	Data	Versão	Descrição
Equipe		0.1	Criação do documento e definição da visão geral e escopo.
Equipe		0.2	Elaboração dos requisitos funcionais.
Equipe		0.3	Elaboração dos requisitos não funcionais.
Equipe		0.4	Revisão dos requisitos.
Equipe		0.5	Adequação da arquitetura PWA, (suporte offline, alerta luminoso via Flash e sincronização).
Equipe		0.6	Atualização da infraestrutura da containerização (Dockerfile e Docker Compose) e refinamento do documento.

Sumário

Sumário.....	4
1.Introdução.....	5
Descrição do Projeto.....	5
2. Objetivo Geral.....	5
3. Objetivos Específicos.....	5
4. Justificativa.....	5
4. Público-alvo.....	6
5. Escopo do Projeto.....	6
6. Requisitos Funcionais (RF).....	6
7. Requisitos Não Funcionais (RNF).....	7
8. Tecnologias Utilizadas.....	7
9. Arquitetura do Sistema.....	8
10. Fluxo do Sistema.....	8

1.Introdução e Descrição do Projeto

O EchoSense é uma solução de tecnologia assistiva desenvolvida como uma aplicação web progressiva (PWA - Progressive Web App) voltada para pessoas com deficiência auditiva. O sistema identifica sons importantes do ambiente (como campainhas, alarmes de emergência e choro de bebê) utilizando o modelo de inteligência artificial YAMNet (TensorFlow.js) executado diretamente no navegador móvel do smartphone. Ao identificar um som relevante, o sistema emite um alerta visual, aciona a lanterna/flash do celular e envia os dados para uma API REST desenvolvida em Spring Boot, que registra os eventos em um banco de dados PostgreSQL e disponibiliza o histórico de notificações.

2. Objetivo Geral

Desenvolver um ecossistema composto por uma PWA (React) e uma API REST (Spring Boot + PostgreSQL) capaz de capturar, classificar, armazenar e disponibilizar eventos sonoros identificados por inteligência artificial, promovendo acessibilidade e segurança por meio de alertas visuais e luminosos no celular.

3. Objetivos Específicos

- Desenvolver uma API REST utilizando Java e Spring Boot para processamento, validação e persistência de dados.
- Estruturar o front-end em React como PWA, permitindo o uso em navegadores móveis (ex: Google Chrome no smartphone) sem necessidade de app nativo.
- Integrar a aplicação ao modelo de IA YAMNet para execução da classificação de áudio localmente no cliente (Edge Computing).
- Garantir o funcionamento da detecção e emissão de alertas mesmo quando o celular estiver offline, com sincronização posterior na API.
- Implementar alertas luminosos acionando o Flash/LED do smartphone através de Web APIs.
- Armazenar o histórico de eventos sonoros no banco PostgreSQL.
- Utilizar Dockerfile e Docker Compose para containerização e facilitação da implantação da aplicação e do banco de dados.

4. Justificativa

Pessoas com deficiência auditiva enfrentam grandes desafios cotidianos para perceber alertas sonoros essenciais. Situações como um alarme de incêndio, uma campainha ou o choro de um bebê demandam atenção imediata. O EchoSense atua como uma solução de alto impacto ao converter esses estímulos sonoros em alertas visuais de alto contraste e pulsos luminosos (flash) no smartphone. Por ser uma PWA, elimina barreiras de instalação e funciona sem dependência contínua de internet, maximizando a autonomia e a segurança do usuário.

5. Público-alvo

- Pessoas com deficiência auditiva profunda, severa ou parcial.
- Pessoas idosas com perda progressiva da audição.
- Instituições interessadas em soluções de acessibilidade e inclusão.

6. Escopo do Projeto

Escopo Negativo (Fora do Escopo)

- O sistema não realizará o treinamento do modelo de IA (utilizará o modelo YAMNet pré-treinado).
- O sistema não fará transmissão de áudio em tempo real (streaming) para o servidor.
- O sistema não exigirá publicação em lojas de aplicativos (Google Play ou App Store).

Escopo Positivo (Dentro do Escopo)

- Captura contínua de áudio do ambiente via Web Audio API no navegador do celular.
- Processamento e classificação dos sons via YAMNet em TensorFlow.js no cliente.
- Disparo de alertas visuais na tela e acionamento do Flash/LED do smartphone.
- Armazenamento temporário de eventos no navegador (IndexedDB) durante momentos offline.

- Envio de requisições JSON para a API Spring Boot quando houver conexão com a internet.
- Cadastro e autenticação segura de usuários (JWT).
- Consulta e gerenciamento (exclusão) do histórico de eventos detectados.

7. Requisitos Funcionais (RF)

RF01 – O sistema deve permitir o cadastro de novos usuários.

RF02 – O sistema deve permitir a autenticação de usuários e geração de token de acesso (JWT).

RF03 – O sistema deve capturar o áudio do ambiente por meio do microfone do dispositivo móvel.

RF04 – O sistema deve identificar eventos sonoros relevantes utilizando o modelo YAMNet.

RF05 – O sistema deve acionar um alerta luminoso (Flash/LED do smartphone ou piscar a tela) ao detectar um som crítico.

RF06 – O sistema deve enviar as informações da detecção (tipo de som, nível de confiança e data/hora) para a API REST.

RF07 – O sistema deve registrar os eventos sonoros no banco de dados PostgreSQL vinculados ao usuário autenticado.

RF08 – O sistema deve permitir a consulta do histórico de eventos sonoros detectados do usuário autenticado.

RF09 – O sistema deve permitir a consulta de um evento específico por seu identificador.

RF10 – O sistema deve permitir a exclusão de eventos cadastrados no histórico pelo próprio usuário.

RF11 – O sistema deve permitir a detecção de sons e o disparo de alertas visuais/ luminosos mesmo quando o dispositivo estiver offline.

RF12 – O sistema deve sincronizar automaticamente com a API os eventos detectados offline assim que a conexão for restabelecida.

RF13 – O sistema deve retornar mensagens estruturadas de sucesso ou de erro (com os devidos códigos HTTP) para as requisições realizadas.

8. Requisitos Não Funcionais (RNF)

RNF01 – O Back-end deve ser desenvolvido em java 21 utilizando o framework Spring Boot.

RNF02 – O banco de Dados relacional utilizado deve ser o PostgreSQL.

RNF03 – O Front-end deve ser desenvolvido em React e estruturado com PWA (Progressive Web App).

RNF04 – O sistema deve utilizar Dockerfile para a criação da imagem API Spring Boot e Docker Compose para orquestrar a execução da aplicação integrada ao banco de dados PostgreSQL.

RNF05 – A API deve seguir uma arquitetura em camadas (Controller, Service, Repository, Model/ Entity, DTO).

RNF06 – A API deve comunicar-se e retornar dados no formato JSON.

RNF07 – O tempo de processamento entre o recebimento do payload na API e a gravação no banco de dados não deve exceder 500ms.

RNF08 – O sistema deve disponibilizar documentação interativa da API via Swagger/ OpenAPI(Springdoc).

RNF09 – O controle de acesso e a segurança das rotas da API devem ser realizadas com Spring Security e Tokens JWT.

9. Tecnologias Utilizadas

- Back-end: Java 21, Spring Boot (Spring Web, Spring Data, Spring Security, Validation).
- Banco de Dados: PostgreSQL
- Front-end/ PWA: [React.js](#), Web Audio API, Service Workers, IndexedDB, Bootstrap/ Tailwind.
- Inteligência Artificial: [TensorFlow.js](#) com Modelo YAMNet.
- Containerização: Docker (Dockerfile) e Docker Compose.
- Documentação de API: OpenAPI 3/ Swagger UI.

10. Arquitetura do Sistema e Padrões de Projeto

A. Arquitetura do Sistema

O sistema adota uma abordagem de Edge Computing com API REST:

1. PWA / Client-side: O smartphone executa a inteligência artificial (YAMNet) de forma local no navegador. Isso evita o envio de streaming contínuo de áudio para o servidor, garantindo baixa latência, economia de dados e privacidade.

2. API Spring Boot / Server-side: A API atua como receptora de metadados dos eventos (ex: tipo de som, confiança e horário), validando o acesso do usuário e gravando as informações no PostgreSQL.

B. Padrões de Projeto (A definir)

11. Fluxo do Sistema

1. O usuário acessa a aplicação PWA no smartphone via navegador (ex: Google Chrome) e realiza o login.
2. A PWA solicita permissão de uso do microfone e da câmera (para controle do Flash/LED).
3. O Service Worker baixa os recursos e o modelo YAMNet para o armazenamento local do celular.
4. O sistema realiza a escuta contínua e classificação do áudio ambiente em tempo real no próprio celular.

5. Ao reconhecer um som relevante (ex: Choro de Bebê com índice de confiança acima do limite definido):

- Fluxo A (Alerta): A PWA exibe imediatamente a notificação visual na tela e faz o flash do celular piscar.
- Fluxo B (Se Online): A PWA dispara um POST /api/v1/eventos para a API Spring Boot, salvando o evento no PostgreSQL.
- Fluxo C (Se Offline): A PWA guarda o registro no banco local do navegador (IndexedDB).

6. Ao recuperar a conexão com a internet, o Service Worker sincroniza automaticamente os registros acumulados no IndexedDB com a API PostgreSQL.